Crosslink Guide

Two confirmation guarantees from one transaction chain

m@rek.onl

Abstract

A proof-of-work chain can keep accepting blocks when a voting committee cannot communicate. A voting protocol can refuse to make conflicting decisions even during a partition. Crosslink combines these different guarantees without changing the order of transactions inside the underlying chain. This volume constructs the exact Crosslink 2 state machine at one formal source pin: its two kinds of blocks, cross-links, validity rules, persistent finalised view, and bounded-available view. It proves the elementary safety implications of those rules, while keeping the assumptions about the two component protocols and their progress explicit.

Contents

1 Two ledgers and their guarantees

1.1 The payment question

A recipient who sees a payment in a recently mined block faces two questions: is the block valid, and will the network later replace its branch? *Consensus Guide*, §*Work, difficulty, and chain selection*, constructs the proof-of-work rule and cumulative-work comparison. §*Reorganisation and confirmation confidence* explains why a finite confirmation depth gives conditional confidence, not an unconditional promise against every possible future execution.

Crosslink supplies two views of the *same kind of transaction chain*. A *finalised view* advances only when the construction's finalisation condition is met. A *bounded-available view* may include a confirmed tail beyond that point. If finalisation does not advance for too many blocks, the design restricts which new blocks may carry ordinary user activity.

The name does not mean that every request is always answered within a fixed time. The bound is on a specified block-height gap. Voting can remain stalled indefinitely, and restricted blocks can still continue to be produced.

1.2 The selected construction

The endpoint is the Crosslink 2 formal construction at `fe6e1d6`. It combines a best-chain protocol, denoted Π_b, and a voting agreement protocol, denoted Π_f. These are parameterised interfaces with explicit requirements; the source does not specify one production-ready byte encoding, staking system, and voting implementation for all of them.

The elementary implications proved here concern the *modified* component protocols actually participating in Crosslink. Transferring assumptions from an unmodified protocol or from a different prototype requires an additional argument. In particular, the separately identified v13 implementation in the last section is not silently treated as this formal state machine.

1.3 Executions, honesty, safety, and progress

An *execution* is a history of message sends, message deliveries, local computations, and decisions, starting at protocol initiation. We index events by times $t \geq 0$. A complete history fixes which events occurred, even if the protocol originally chose some of them randomly.

A node is *honest at time t* when it has followed its specified rules throughout the history up to and including t. It is therefore also honest at every earlier time. This convention is essential when a proof refers to an earlier event at which a node stored its current finalisation point. Merely behaving correctly for one recent instant would not satisfy the definition.

A *safety property* says that a forbidden outcome never occurs; for example, two honest nodes never finalise conflicting chains. A *liveness property* says that progress eventually occurs under stated conditions; for example, a transaction is eventually included or a finalisation point eventually advances. These are different assertions. A protocol that makes no decisions can preserve agreement while failing to make progress.

The default communication model permits arbitrary message delays and losses. Required messages are authenticated, meaning their content is attributed to the actual sender under the signature assumptions in *Crypto Guide*, §*Security contracts and reductions*. A *network partition* is an interval during which some groups of nodes cannot exchange the relevant messages. No bounded-delivery assumption is smuggled into a safety proof.

Progress arguments may require intervals in which messages arrive within a delay Δ, sufficiently many honest producers participate, and sufficiently much voting weight is online. Those conditions must cover the actual interval and state in which progress is claimed. A timing argument for one successful interval does not automatically cover every later interruption and recovery.

1.4 Chains, prefixes, and compatibility

A chain is a sequence of blocks beginning with a distinguished genesis block. Each later block contains a collision-resistant hash reference to its parent. Subject to the hash-binding assumption from *Crypto Guide*, §*Authenticated trees and append-only histories*, its tip determines a unique ancestor sequence. We therefore use the same symbol for a tip and its chain.

Write $A \preceq_b B$ when the best-chain block sequence A is a prefix of B, including equality. Write

$$A \sim_b B \quad \iff \quad A \preceq_b B \text{ or } B \preceq_b A$$

for *compatibility*. They conflict when neither is a prefix of the other. Use subscripts f for voting-chain blocks. Subscripts may be omitted when the chain type is unambiguous.

Two prefixes of one chain are compatible: compare their lengths; the shorter occupies the same initial positions of that chain. We will repeatedly use this simple fact.

A sequence $A_0, A_1, \ldots$ is *monotone by extension* if $A_j \preceq A_k$ whenever $j \leq k$. Pairwise compatibility is weaker. The sequence

$$[g, a, b], \quad [g, a]$$

contains compatible chains but retracts the last block. Thus agreement between a node's views at two times does not, on its own, prove that it never rolls its own view back.

The height of genesis is zero. H^{-k} means remove k blocks from the tip, stopping at genesis if necessary. The tip has depth zero; its parent has depth one. A node's current best-chain tip is C_i^t, with i the node and t the time. This function of node and time is its *view*. A view is local knowledge, not an omniscient global list of all blocks.

2 Chain and agreement interfaces

2.1 The best-chain component

A best-chain block is valid only if its parent is valid and it satisfies the inherited block and transaction rules. Its header summarises and commits to the block. Header validity is necessary but not sufficient for full block validity: a header with valid proof of work may still commit to an invalid transaction.

Each valid chain has a score in a totally ordered set: any two scores can be compared, and the comparisons are consistent. Extending a chain by one valid block strictly increases its score. For proof of work, the score is cumulative work, not merely height. The node selects a maximum-score valid chain it knows, with a specified tie-break rule.

The Crosslink validity additions restrict the set of valid chains. They do not replace the score comparison with a preference for whichever branch has the most attractive voting certificate. Any claimed security of this modified process must account for the validity restrictions as well as the unchanged score function.

2.2 Two chain-security properties

Fix a positive integer confirmation depth σ. *Prefix Agreement at depth* σ means that for all honest observations, at possibly different times,

$$(C_i^t)^{-\sigma} \sim_b (C_j^u)^{-\sigma}.$$

The entire execution is quantified over. Agreement only at the same instant would allow every node to switch together between two incompatible histories on alternating days.

Prefix Consistency at depth σ is the directional condition

$$t \leq u \quad \Longrightarrow \quad (C_i^t)^{-\sigma} \preceq_b C_j^u$$

for nodes honest at their respective observation times. The earlier confirmed prefix must still be inside the later *full* best chain.

Prefix Consistency implies Prefix Agreement at the same depth. Order the two times, say $t \le u$. By consistency the earlier truncated chain is a prefix of C_j^u; the later truncated chain is also a prefix of C_j^u. The two are therefore compatible. This argument does not say that the later truncated chain must be at least as long as the earlier one.

These are properties of an execution, not automatic facts about every proof-of-work network. One can subsequently assign a failure probability under a mining and communication model. Long partitions, censorship of a node's block information, or an adversarial work advantage can invalidate the premises.

2.3 Voting epochs, proposals, and certificates

The agreement component proceeds through numbered *epochs*. An epoch is a voting opportunity with a designated proposer and a fixed assignment of voting units to participants. A participant may control several units. Let the total number of units in epoch e be $n_e > 0$. Weights can equivalently be represented by nonnegative amounts and summed; the counting argument below uses the integer-unit model.

A *proposal* identifies its epoch, its parent voting block, and its payload. The proposer signs all of this content. A *ballot* is an authenticated vote for that precise proposal. A *certificate*, called a notarisation proof by the formal construction, demonstrates that distinct voting units of total weight at least $2n_e/3$ supported it, together with any extra conditions imposed by the component protocol.

A direct certificate implementation could list each participating identifier and its valid signature and add its known weight, rejecting duplicate identifiers or units. Compact certificate schemes may replace that list only if their verified statement establishes the same property. A count asserted inside a payload is not such a proof.

The resulting voting block consists of the proposal and its certificate, signed again by the proposer. The parent hash of the next proposal commits to the *whole* parent voting block, including its certificate and outer signature. The formal model requests strong signature unforgeability: without the key an adversary cannot create a new accepted signature even for a message already signed, as defined in *Crypto Guide*, §*Security contracts and reductions*.

Proposal validity and block validity are distinct. A valid proposal has no certificate yet. A valid voting block also requires its certificate, its outer signature, and a valid parent chain. An honest proposer emits at most one proposal and one corresponding signed block for its epoch. A dishonest proposer may produce different signed representations of the same certified proposal; uniqueness of the proposal does not make the entire block encoding unique.

2.4 The one-third condition and what it proves

A voting unit is cast non-honestly if it is used without its holder's authorisation, used for two distinct proposals in an epoch, or used contrary to the honest-voting rules. The selected condition is:

for each epoch e, fewer than $n_e/3$ units are ever cast non-honestly.

The word *ever* includes later use of an old key to forge historical ballots. Deleting an expired signing key can therefore be part of meeting the assumption. A majority of currently honest machines does not imply that all their historical keys remain safe.

Suppose two distinct proposals for epoch e each have sound certificates, supported by sets S, S'. Then

$$|S \cap S'| = |S| + |S'| - |S \cup S'| \ge \frac{2n_e}{3} + \frac{2n_e}{3} - n_e = \frac{n_e}{3}.$$

Under the strict one-third bound, the intersection contains an honestly cast unit. Such a unit cannot vote for two distinct proposals, a contradiction. Hence at most one proposal in that epoch can obtain a valid certificate.

For $n_e = 10$, a certificate needs at least seven units and two certificates intersect in at least four. The strict bound permits at most three non-honest units. Rounding strengthens the intersection, but does not change the need for the strict bound in the general statement.

This proves *same-epoch proposal uniqueness*. It does not prove agreement between decisions formed in different epochs. That needs the component's parent-selection, honest-voting, and finality rules. Nor does the argument prove that any certificate will ever be assembled.

2.5 Finality is evaluated in a context

The voting component supplies an efficiently computable function $F(B)$, the last voting block final in the context of B. It has the following contract:

$$\begin{gathered} F(B) = \bot \iff B \text{ is invalid,} \\ F(B) \preceq_f B \quad \text{for valid } B, \\ B \preceq_f B' \implies F(B) \preceq_f F(B'), \\ F(g_f) = g_f. \end{gathered}$$

Here g_f is the voting genesis and $\bot$ denotes failure. The third rule says that extending a valid voting chain does not retract its contextually final prefix.

For a concrete illustration of such a function, the adapted Streamlet finality rule examines a valid voting chain for three adjacent blocks with consecutive epoch numbers. The middle block of each such triple is final in that context; $F(B)$ returns the latest such middle block, or genesis if no triple exists. Extending the chain preserves old triples, so this function has the required monotonicity. This illustrates the function, not an imported proof that every Crosslink instantiation of Streamlet satisfies all the remaining voting and timing assumptions.

Final Agreement is the component security premise: for all valid voting blocks B, B' observed at times when their observers are honest,

$$F(B) \sim_f F(B').$$

A block is not final merely because its payload calls it final. The context and the verified function F supply the evidence.

3 Cross-links and the state machine

3.1 Parameters and cross-linked objects

Choose $\sigma \geq 1$ for finalisation confirmation depth and a finality-gap bound L substantially larger than σ; the formal design recommends $L \geq 2\sigma$. For the bounded-available view a client chooses $1 \leq \mu \leq \sigma$, normally $\mu = \sigma$. These are block counts, not time durations.

Each best-chain header H gains a field $\mathsf{ctx}(H)$ committing to a voting block. Each non-genesis voting proposal and block B carries a list of exactly σ best-chain headers, in ancestor-to-descendant order:

$$\mathsf{tail}(B) = [H_{h-\sigma+1}, \ldots, H_h].$$

Thus the first list entry has greatest depth relative to its tip. The snapshot is its parent:

$$S(B) = \mathrm{parent}(\mathsf{tail}(B)[0]) = H_{h-\sigma}.$$

It is neither the first supplied header nor the last one.

Set $S(g_f) = g_b$, where g_b is best-chain genesis, and $\mathsf{ctx}(g_b) = g_f$. The voting genesis carries a special empty tail, not a fabricated σ-header list. The explicit genesis convention avoids trying to manufacture two ordinary cryptographic hashes that refer cyclically to each other.

For any valid best-chain block H, define

$$\mathsf{LF}(H) = F(\mathsf{ctx}(H)), \qquad Q(H) = S(\mathsf{LF}(H)).$$

The first is a voting block; the second is a best-chain block. Keeping these types distinct prevents a height in one chain from being compared accidentally with a height in the other.

3.2 Voting-chain validity

A non-genesis voting proposal or voting block must satisfy its inherited proposal or block rules, including parent validity. It additionally satisfies:

1. *Snapshot linearity:* $S(\text{parent}(B)) \preceq_b S(B)$.
2. *Tail confirmation:* its supplied σ headers form the tail of a fully valid best-chain block sequence.

Full validity is important. Checking only header hashes, targets, and proof of work does not validate the transactions underneath. The header list tells a validator what additional blocks it must obtain; it does not excuse obtaining and validating them.

Snapshot equality with the parent is allowed. A voting chain can therefore keep making valid proposals about the same snapshot while the best-chain snapshot is temporarily unable to advance. Along any valid voting chain, snapshot linearity implies

$$B \preceq_f B' \quad \Longrightarrow \quad S(B) \preceq_b S(B')$$

by induction over the intervening parent links.

These are objective validity rules: every observer with the same objects and ancestor data obtains the same result. A certificate with all voting weight behind it cannot make an objectively invalid snapshot chain valid.

3.3 Best-chain validity

Each non-genesis best-chain block H, with parent P, must satisfy its inherited block and transaction rules and:

1. $\mathsf{ctx}(H)$ is a valid voting block.
2. $\mathsf{LF}(P) \preceq_f \mathsf{LF}(H)$.
3. $Q(H) \preceq_b H$.
4. With
$$d_{\text{fin}}(H) = \text{height}(H) - \text{height}(Q(H)),$$
either $d_{\text{fin}}(H) \le L$, or $\mathsf{Stalled}(H)$ is true.

The predicate Stalled specifies the permitted restricted blocks during a long finalisation gap. It is an application-level parameter of this formal construction; it must be fixed as an objectively checkable block-validity rule.

The third rule makes the height difference meaningful: the snapshot is an ancestor, not an unrelated block at a smaller height. The gap depends on the block's committed context, not on whichever finalisation point one observer currently stores.

The second rule is about the *last final ancestor* of the contexts. It does not require the raw contexts themselves to form one monotonically extending voting chain along every best-chain branch.

3.4 Honest proposal, voting, and production

An honest voting proposer waits until it knows a best chain with at least $\sigma + 1$ blocks including genesis. It tries to use that chain's last σ headers. If the resulting snapshot would violate snapshot linearity relative to the proposal's parent, it repeats the parent's header list. It otherwise follows the component's proposer and parent-selection rules.

An honest voter first validates the offered header tail and its full blocks, updating its views of both components. Each best-chain block may refer to a voting context, whose ancestors and snapshot blocks must in turn be validated. Already validated objects can be cached by their hashes. The resulting dependency graph must be resolved to known valid ancestors; an unresolved cycle is not validation evidence.

After updating its view, the voter votes only if the proposal is objectively valid *and*

$$S(B) \preceq_b (C_i^t)^{-\sigma}.$$

It also obeys every inherited voting condition. The offered tail need not be identical to the voter's eventual best-chain tail: both can extend the same snapshot on different short branches. The snapshot itself must be σ-confirmed in the voter's updated best chain.

An honest best-chain producer retains the maximum-score rule for choosing its best-chain view. For a proposed new block with parent P, the formal context-selection rule considers valid voting-chain tips T satisfying

$$\mathsf{LF}(P) \preceq_f F(T).$$

It selects a longest such voting chain; ties maximise the best-chain score of $S(F(T))$, then choose the smallest hash. The resulting block must still satisfy *all* best-chain validity rules, including snapshot ancestry.

In particular, this ranking does not authorise attaching a foreign-fork snapshot to an incompatible best-chain parent. Proving that the ranking and the complete validity conditions always permit the desired progress is a liveness obligation. The simple existence of some valid context, shown later, is not by itself a proof about this ranking in every reachable state.

4 Ledger extraction and safety

4.1 Computing a candidate

For two chains, their *last common ancestor* is the longest prefix they share. It can be found by following parent links until the histories meet. Define

$$N(H) = \mathsf{LCA}_b\big(Q(H), H^{-\sigma}\big).$$

By construction,

$$N(H) \preceq_b Q(H), \qquad N(H) \preceq_b H^{-\sigma}.$$

Because block validity already gives $Q(H) \preceq_b H$, the two arguments lie on one chain. Their last common ancestor is simply the shorter one. Writing the operation explicitly keeps the ancestor requirement visible.

Why not use $Q(H)$ directly? The snapshot in a committed voting context need not be σ blocks deep relative to this particular best-chain block H. The candidate must carry both forms of evidence: it lies in the final snapshot and in the σ-confirmed prefix of the local best chain.

4.2 The persistent finalised view

Node i stores f_i, initially best-chain genesis. Whenever its best-chain view changes to H, it performs:

1. Compute $N = N(H)$.

2. If $f_i \preceq_b N$, set $f_i \leftarrow N$.
3. Otherwise retain f_i. If $N \not\preceq_b f_i$, also record a finalisation safety hazard, with the conflicting chains and the relevant history of stored updates.

The superscript f_i^t denotes the stored value after the events through time t.

A candidate that is merely an ancestor of the stored point causes no rollback and no fork-conflict report. An incompatible candidate causes a report but still no rollback of the stored finalised view. This is not a rule to ignore the evidence or continue ordinary application operations after a hazard.

Local monotonicity. Every change to f_i is an extension. Induction on updates therefore proves

$$t \le u \quad \Longrightarrow \quad f_i^t \preceq_b f_i^u$$

while the node follows the update algorithm. This is an exact state-machine property, with no voting threshold or honest-work assumption.

It does not prove that another node stored a compatible chain. Two disconnected nodes can each extend different chains monotonically. Global agreement needs a separate premise.

4.3 The bounded-available view

For a selected depth $1 \le \mu \le \sigma$, define

$$a_{i,\mu}^t = \begin{cases} (C_i^t)^{-\mu}, & f_i^t \preceq_b (C_i^t)^{-\mu}, \\ f_i^t, & \text{otherwise.} \end{cases}$$

This immediately gives the local ledger-prefix property

$$f_i^t \preceq_b a_{i,\mu}^t.$$

The otherwise branch matters. A maximum-work reorganisation can yield a shorter chain in block count, or a context whose candidate temporarily retreats along the same history. The application must not see its bounded-available ledger retract behind its already stored finalised prefix.

The bounded-available view is not generally monotone. Its unfinalised tail can be replaced. Reducing μ can reveal more recent blocks, but it also changes the chain-security assumption needed to protect that tail. It does not change f_i, whose candidate always uses the fixed σ.

4.4 A local update example

Take $\sigma = \mu = 2$. Suppose the validated local inputs have the following ancestry relations. The prime marks a new best-chain branch; all listed snapshots remain on their corresponding best chains, and both branches agree through height seven.

	C_i	$Q(C_i)$	$(C_i)^{-2}$	$N(C_i)$	f_i ; $a_{i,2}$
t_1	H_8	H_5	H_6	H_5	H_5 ; H_6
t_2	H'_9	H_4	H_7	H_4	H_5 ; H_7
t_3	H'_{10}	H_7	H'_8	H_7	H_7 ; H'_8

At t_2, the candidate drops from height five to height four, but the stored finalised point stays at five. At t_3 it advances to seven. This table illustrates the local algorithm on already validated inputs; it is not a complete generated voting-and-mining execution.

Now suppose a later candidate lies on a fork that diverges before H_5. Neither it nor H_5 is a prefix of the other. The algorithm retains its stored finality and records a hazard. The agreement theorems below identify which component premises must have failed for honest nodes to reach such incompatible finalisation evidence.

4.5 Two historical-witness lemmas

For every honest observation of f_i^t, there is an earlier time $r \le t$ such that

$$f_i^t \preceq_b (C_i^r)^{-\sigma}.$$

Choose the last time when the stored value changed. At that time it was assigned a candidate, and every candidate is a prefix of the then-confirmed best chain. If it never changed, choose time zero and the genesis chain.

The same argument also supplies a voting context observed honestly at some such earlier time, say B_i, with

$$f_i^t \preceq_b S(F(B_i)).$$

At the assignment time $B_i = \mathsf{ctx}(C_i^r)$, and the candidate was a prefix of $Q(C_i^r) = S(F(B_i))$. Again genesis handles the no-update case.

These lemmas are different. The first gives best-chain confirmation evidence. The second gives voting-finality evidence. Replacing one by the other without its corresponding premise would not establish the desired safety theorem.

The historical times are legitimate honest observations because honesty at t includes following the rules at all earlier times. The stored value may now be deeper than a current truncated view; the lemma does not falsely insist that its witness time is t.

4.6 Safety from the best-chain component

Theorem. If the modified best-chain component has Prefix Agreement at depth σ, then every pair of honest finalised views, at arbitrary times, is compatible:

$$f_i^t \sim_b f_j^u.$$

Proof. Use the first historical-witness lemma to obtain

$$f_i^t \preceq_b (C_i^r)^{-\sigma}, \qquad f_j^u \preceq_b (C_j^s)^{-\sigma}.$$

Prefix Agreement makes the two right-hand chains compatible. Choose the longer of them. Both finalised views are prefixes of that one chain, so they are compatible. No Final Agreement assumption about the voting component was used.

4.7 Safety from the voting component

Theorem. If the modified voting component has Final Agreement, then every pair of honest finalised views is compatible.

Proof. Use the second historical-witness lemma:

$$f_i^t \preceq_b S(F(B_i)), \qquad f_j^u \preceq_b S(F(B_j)).$$

Final Agreement gives $F(B_i) \sim_f F(B_j)$. Snapshot linearity along a valid voting chain therefore makes their snapshots compatible. Choose the longer snapshot. Both finalised views are prefixes of it, proving compatibility. No Prefix Agreement assumption about the best-chain component was used.

These are proofs about the defined state machine and its stated component premises. They do not fill missing proofs that the modified components meet those premises under a particular resource distribution, network schedule, or implementation.

4.8 What the disjunction means

Let P be the event that the modified best-chain execution has Prefix Agreement at depth σ, and F the event that the modified voting execution has Final Agreement. Let E mean a conflict between honest finalised views. The two theorems give

$$E \subseteq P^c \cap F^c.$$

A conflict therefore requires both safety premises to fail. Their failure is necessary, not sufficient: violating an assumption does not force a successful attack.

For any probability model on executions,

$$\Pr[E] \leq \Pr[P^c \cap F^c] \leq \min\{\Pr[P^c], \Pr[F^c]\}.$$

Multiplying the two failure probabilities would additionally require independence or another justified joint bound. The protocols interact and may share adversaries, so independence is not supplied by their different names. If cryptographic validity can also fail outside the model, its failure event must be included separately.

The property is often called assured finality. Here it means the precisely quantified compatibility statement above. Combined with the independent local monotonicity proof, it gives both no local retraction and no cross-node conflict under either component safety premise.

4.9 The bounded-available tail has a different guarantee

For every honest observation and $\mu \leq \sigma$, some earlier time $r \leq t$ satisfies

$$a^t_{i,\mu} \preceq_b (C^r_i)^{-\mu}.$$

If the available view uses the current truncated chain, choose $r = t$. Otherwise it equals f^t_i, and the first historical lemma applies because a σ-truncated chain is a prefix of its μ-truncation.

It follows, by the same two-prefix argument, that Prefix Agreement of the best-chain component at depth μ implies agreement between all bounded-available views at that depth.

Under the stronger directional Prefix Consistency at depth μ, one can additionally prove, for $t \leq u$,

$$a^t_{i,\mu} \preceq_b C^u_j.$$

Apply consistency to its historical witness at $r \leq t \leq u$. This says the earlier available view is in the later full best chain. It does not say it is contained in every later μ-truncated view.

Voting Final Agreement alone protects the finalised prefix, not every block in the available tail. Nevertheless, each local available view always extends its local finalised prefix. When two finalised views are compatible, their shorter prefix is therefore inside both nodes' available views. No stronger tail guarantee follows from the voting theorem alone.

4.10 Why transaction order is preserved

Both outputs f^t_i and $a^t_{i,\mu}$ are actual best-chain block sequences. Extract the transaction ledger by reading blocks from genesis and transactions in each block's committed order. A prefix of blocks gives a prefix of that transaction sequence.

There is no operation that takes transactions from incompatible branches, reorders them, and filters out the ones that have become invalid. Block heights, note-tree positions, nullifier sets, transaction expiry, and branch-local state evolution retain their inherited meaning on the chosen chain.

This is why the block-level proofs transfer cleanly to transaction prefixes. They still require full validation of those blocks. An authenticated reference to a snapshot without its valid underlying ledger is not enough.

5 Progress, stake, and settlement

5.1 Restricted blocks do not mean a stopped block chain

A stalled-block policy is a predicate on block content. One possible policy permits only a coinbase transaction, which creates and distributes that block's authorised subsidy without spending previous outputs. The formal construction leaves the exact policy to the transaction protocol. It is not correct to assume that every transaction type is automatically harmless merely because it is not an ordinary payment.

Assume that for every valid best-chain parent P, a producer can construct a valid coinbase-only child satisfying Stalled. Keep the parent's voting context:

$$\mathsf{ctx}(H) = \mathsf{ctx}(P).$$

Then context validity is inherited, $\mathsf{LF}(H) = \mathsf{LF}(P)$, and the extension rule holds. Also $Q(H) = Q(P) \preceq_b P \preceq_b H$. If the gap exceeds L, the stalled predicate permits the child.

This proves the existence of content satisfying the extra Crosslink validity rules. It does not manufacture proof of work: the inherited mining process still has to find a header meeting the required target. Under its usual progress conditions, that process can keep increasing the chain's work even though user spends are restricted.

The distinction matters. If block production itself stopped, there would be no further work accumulating above the tail. Stalled Mode is designed to restrict new economic exposure while allowing the chain's work process to continue.

5.2 What the gap bound actually bounds

Suppose along a valid branch the contextually final snapshot stays fixed at a block Q of height h_Q. For every later block that is *not* a stalled block,

$$\mathrm{height}(H) - h_Q \leq L.$$

There are therefore at most L such block positions after Q while that snapshot remains fixed. Blocks above height $h_Q + L$ must satisfy the restricted policy.

This is an exact consequence of validity. It does not bound the number of later restricted blocks, the wall-clock duration of a stall, or the number of transactions that users may keep constructing and broadcasting off-chain. It also does not guarantee detection of every attack on voting. An attacker who can keep final snapshots advancing may avoid triggering the gap condition.

A crude time estimate might multiply a block count by an expected inter-block interval. An expectation is an average under a probability model, not a deterministic deadline. The selected construction supplies no universal formula turning L into a maximum number of seconds before recovery.

5.3 Why voting can still progress without a new snapshot

Snapshot equality is valid, so a proposer can reuse its parent's valid header tail. This supplies a proposal satisfying the two new objective snapshot rules even when no advancing snapshot is currently available.

It does not ensure that enough honest voters will support it. They must still see the snapshot as σ-confirmed in their updated best-chain views, meet the inherited voting rules, and exchange the required messages. A valid but permanently withheld proposal does not help liveness.

Advancing finalised snapshots needs more:

1. The best-chain component continues growing on a history that honest participants can learn and validate.

2. Enough voting weight sees an eligible, sufficiently confirmed snapshot, and the proposer/ballot/certificate schedule makes progress under the selected voting protocol.

3. A resulting final voting context is incorporated into valid best-chain blocks and becomes visible through their selected maximum-score chains.

4. The candidate extends the stored local finalised prefix and obtains its required best-chain confirmation depth.

If any stage stalls, the later stages cannot simply declare the missing evidence to exist.

The formal source's general liveness discussion does not supply a complete quantitative theorem for these interacting conditions. In particular, a proof that L is almost never reached under specified healthy conditions needs bounds on message delay, certificate production, honest voting weight, honest block production, and their interaction. No single inherited confirmation estimate substitutes for that proof.

5.4 What a stake-based instantiation must supply

The formal voting interface assumes that the voting-unit distribution for an epoch is fixed and known. A *stake-based* instantiation would derive that distribution from funds assigned to participation. Its *roster* is the resulting list of voting identities and weights.

That word does not fill in the required state machine. A concrete instantiation must specify:

1. Which accepted ledger state fixes the roster for each epoch, and how every node authenticates the same state.

2. Which transactions register an identity, assign or remove weight, and transfer any associated spending authority.

3. When a change becomes active and when previously assigned funds can be withdrawn.

4. Which exact epoch and roster identifiers are signed in proposals and ballots, and how signatures are checked against that roster.

A weight cannot be counted twice merely because its owner appears under two identifiers. A newly observed stake transaction on an unsettled branch cannot silently redefine the denominator n_e for an already running epoch.

A delayed withdrawal rule may support an economic penalty for provable misbehaviour. Such a penalty is not identical to cryptographic safety: unavailable funds, uncaught misconduct, or rewards do not change the set-intersection proof by themselves. The formal premise concerns which voting units are ever used non-honestly, including use of old keys.

The selected construction leaves these details as an instantiation boundary. Importing a prototype's bond amount, reward schedule, or delay constant would select an additional protocol whose security and ledger rules would also need to be constructed. Those numbers are therefore not presented as parameters of the formal safety theorems.

5.5 Starting a node and exposing its ledgers

The local variables initially contain genesis, but a newly started node must not represent that as a current settled view. The source recommends a voting checkpoint B_{cp} whose provenance the node trusts, and requires before exposing the ledgers:

$$B_{\mathrm{cp}} \preceq_f \mathsf{LF}(C_i^t),$$
$$S(B_{\mathrm{cp}}) \preceq_b f_i^t,$$

the timestamp of f_i^t is within a configured recency threshold.

A checkpoint is a trusted anchor in history, not a proof that its distribution was honest. A timestamp recency test also depends on the chosen clock and threshold policy. These checks address a startup interface; they do not remove the component security assumptions or prove that an isolated node has found the freshest chain.

The node must durably preserve its finalisation history if applications rely on local monotonicity across restarts. Resetting f_i and exposing a different branch as though no earlier promise had been made would not implement the same continuous application contract. A deployment must specify restoration, checkpoint upgrades, and hazard handling accordingly.

5.6 What a wallet may safely infer

A wallet first needs a valid transaction and a recognised note-tree anchor, as constructed in *Wallet Guide*, §*Constructing and authorising a transaction*. Crosslink changes the confidence attached to a branch prefix; it does not replace note membership or transaction authorisation.

An anchor carried by a block inside a settled prefix remains part of that prefix under the finality premises. But a full validator checks anchor validity against the ancestry of the *candidate block* it is validating, not against an unrelated displayed ledger or a height number alone. If the current best-chain view is transiently inconsistent with a retained finalised prefix, the application must examine that situation; it cannot infer immediate transaction acceptance merely from local finality storage.

Transaction expiry is another independent condition. A transaction may expire while ordinary inclusion is restricted. If the protocol allows non-expiring transactions, or users keep broadcasting fresh transactions during a stall, one cannot conclude that a fixed delay automatically clears every pending transaction.

For a concrete payment, the arithmetic stays the core example: an existing 80 000-zatoshi note funds 50 000 to the recipient, 20 000 change, and 10 000 in fees. If its containing block is only in the bounded-available tail, the recipient is using that tail's chain-security guarantee. When the block enters the finalised prefix, the disjunctive finality guarantee applies. Its proof relation, signatures, amounts, and ciphertexts are unchanged.

6 The implementation boundary

6.1 A pin is not an equivalence proof

The independently pinned implementation reference is `crosslink_monolith` v13 at `807b995`. Its own scope is a prototype. That description alone neither proves nor disproves a particular mechanism, but it prevents using the repository's existence as evidence of a deployed consensus rule set.

Several concrete interfaces show why the formal construction must not be identified with the code merely by matching names. The implementation has a voting payload with a parent reference and a header vector in `librustzcash/zcash_primitives/src/bft.rs`. Its `finalization_candidate` accessor returns the first header. In the formal construction, the snapshot is the *parent* of the first header. These are different objects.

The integration's proposal path selects a candidate height, fetches following headers, and limits how far a candidate may advance in one step. Its finalisation callback takes the first header's block hash, stores a finalised height/hash pair, and requests a state finalisation update. Those operations live in `zebra-crosslink/zebra-crosslink/src/lib.rs`. They are not a literal execution of

$$N(H) = \mathsf{LCA}_b(S(F(\mathsf{ctx}(H))), H^{-\sigma})$$

followed by the persistent extension-only updater.

The voting engine lives in `tenderlink/src/lib.rs` and has its own proposal and multi-stage voting state machine. Its correctness must be analysed against its actual messages, weight accounting, state transitions, and integration callbacks, not inferred from the illustrative three-block finality function given earlier.

These are boundary examples, not a catalogue of historical implementations. This volume's theorems quantify over the formal objects and conditions already defined. Applying them to v13 would require a demonstrated mapping of the concrete execution into that model, preserving the relevant validity, ancestry, honesty, finality, and update properties.

6.2 The obligations of a complete instantiation

A complete implementation of the selected formal endpoint needs the following connected contracts.

1. Canonical encodings, hash domains, signatures, proposal identities, and certificate statements for both block types. All references must bind the intended full object.

2. An objectively validated graph of parent and cross-link dependencies, including full transaction validity and resource limits for fetching and verifying that graph.

3. A voting protocol with a concrete finality function and stated agreement and progress theorems under its actual corruption, message, and roster model.

4. A best-chain validity and production implementation matching the formal rules, together with an argument that its required confirmed-prefix properties hold.

5. Durable ledger extraction, startup criteria, and safety hazard handling that preserve the application's finality contract.

The requirement to validate full blocks is a data-availability requirement in a literal sense: the validator must be able to obtain the data it is asked to validate. A compact hash or certificate is not that data. Caching and validating shared ancestors can save repeated work; they cannot turn unavailable blocks into valid known state.

The safety proofs in this volume are deliberately explicit about their endpoint. They establish that the formal local algorithm converts either of two component agreement properties into compatible finalised transaction histories. They do not claim that a named voting crate, a branch tagged v13, or a particular stake threshold automatically meets every premise.

The construction's central dependency is now visible: valid snapshots move monotonically along voting chains; best-chain blocks commit to a contextually final snapshot; the candidate intersects that evidence with local work confirmation; and the persistent update exposes only extensions. The bounded-available view adds a separately protected tail. Each stage supplies a different guarantee, and the interface between them is part of the protocol.